

## OpenCV 实验大师工具软件 v2.0 使用手册

|                        |    |
|------------------------|----|
| 一：安装与启动.....           | 2  |
| 二： workflow 设计与验证..... | 3  |
| 三：文本处理与识别.....         | 8  |
| 四：YOLO 深度学习训练与部署 ..... | 11 |
| 五：上位机集成部署复用.....       | 14 |
| 六：SDK 集成测试演示 .....     | 15 |

QQ 讨论群：934105010

作者微信



官方微信



## 一：安装与启动

预安装要求：请提前安装好 Python3.11.5 版本，在控制台窗口中输入 `python` 显示如下：

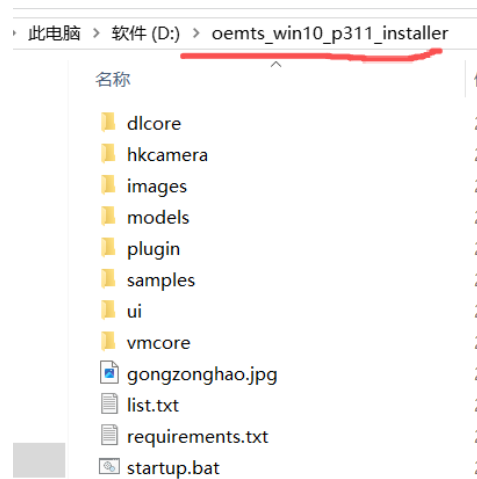
```
C:\Users\Administrator>python
Python 3.11.5 (tags/v3.11.5:cce6ba9, Aug 24 2023, 14:38:34) [MSC v.1936 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
```

上图表示已经满足安装 OpenCV 实验大师的必要前置条件。

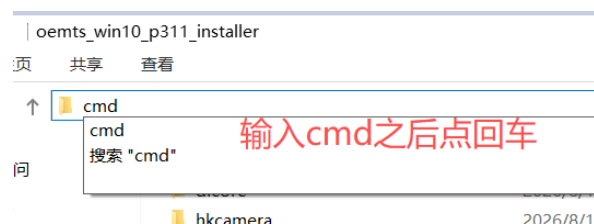
下载 OpenCV 实验大师工具软件 v2.0 版本，浏览器打开下面的下载链接：

<https://share.weiyun.com/tgoxerG9>

可以看到两个安装文件，其中一个为 v1.4.2 版本（不支持模型训练与工作流部署）、另外一个就是 v2.0 版本。点击下载 v2.0 版本，解压缩到 D 盘根目录下（请解压缩到 D 盘根目录下，确保目录跟文件夹名称跟下图一致），如下图所示：



然后在当前文件夹地址栏 空白处输入 `cmd` 然后点回车，截图如下：



这样就打开了 `cmd` 控制台窗口，显示如下（确保起始目录为 `D:\oemts_win10_p311_installer`）：

```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19045.5608]
(c) Microsoft Corporation。保留所有权利。

D:\oemts_win10_p311_installer>
```

然后打开 `requirements.txt` 文件，copy 第四行，请去掉 `#` 符号，粘贴到至打开的控制台窗口，

显示如下：

```
C:\ 管理员: C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19045.5608]
(c) Microsoft Corporation。保留所有权利。
D:\oemts_win10_p311_installer>pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

点击回车，就会开启安装，根据网络下载速度不同大概 3~15 分钟之间完成安装。

安装完成以后，直接在命令行输入：

D:\oemts\_win10\_p311\_installer>startup.bat 回车

就会看到如下的启动界面：



## 二：工作流设计与验证

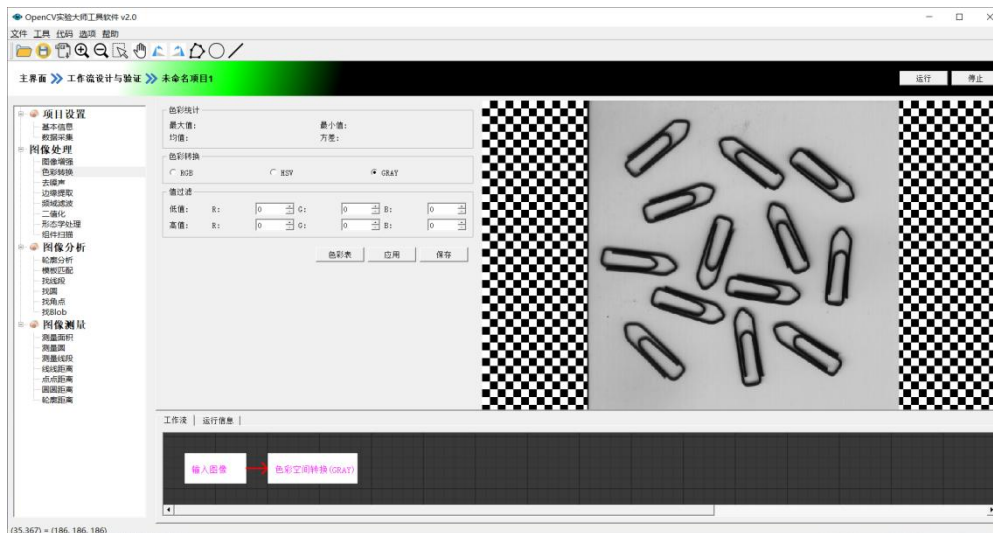
点击【工作流设计与验证】入口，打开界面如下所示：



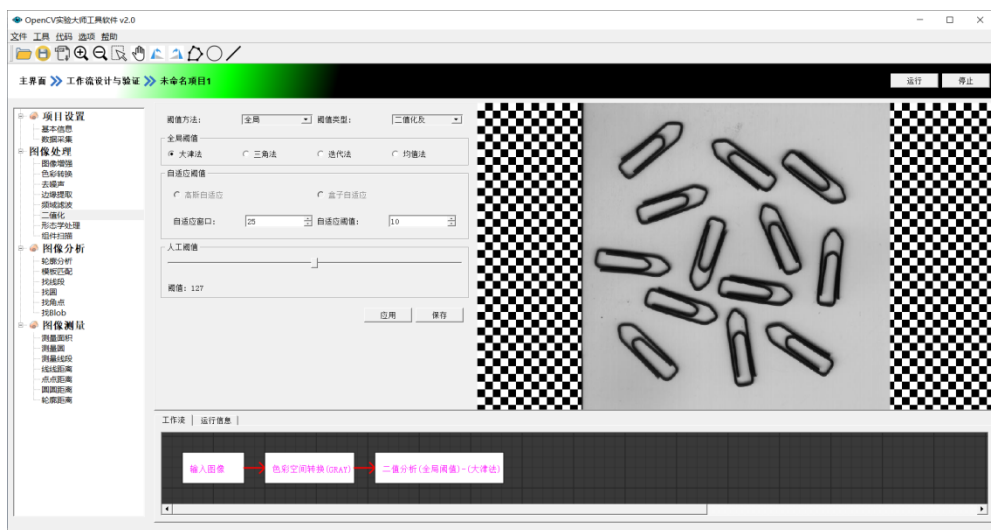
点击数据采集，导入一张图像（**注意：**目录路径不要有空格跟中文），显示如下：



点击左侧算子区域的<色彩转换>，选择 Gray，然后点击【应用】按钮，显示如下（**注意：**底部工作流显示区域已经发生变化）：



继续点击左侧算子区域<二值化>，在参数面板阈值类型 选择 二值化反、然后点击【应用】按钮，显示如下：

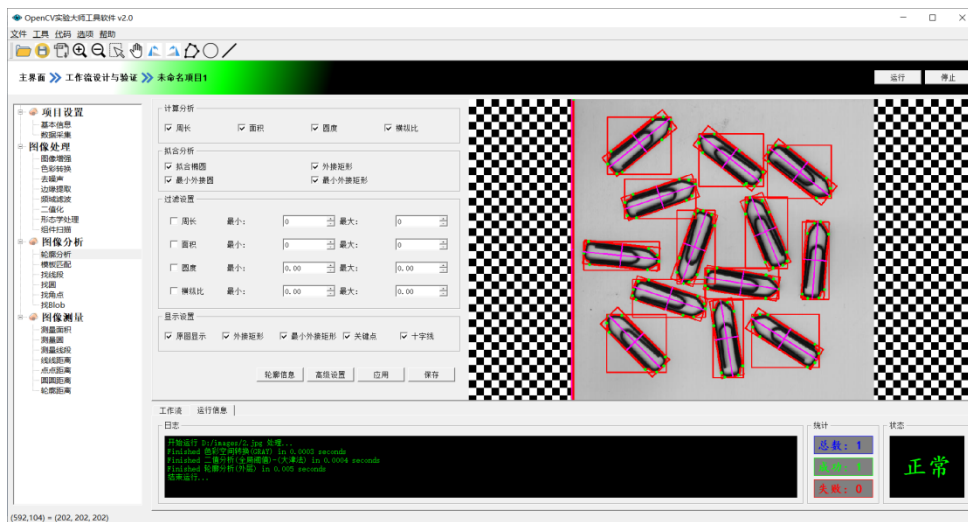


继续点击左侧算子区域<轮廓分析>，在参数面板 显示设置 选中全部 checkbox 打勾、然后

点击【应用】按钮，显示如下：



然后点击右上角【运行】按钮，显示如下：

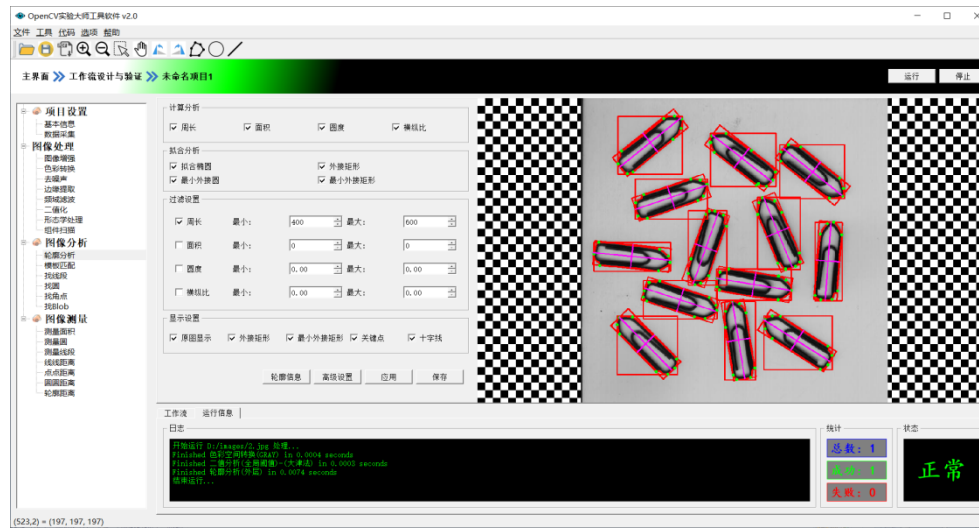


点击轮廓信息按钮，可以查看所有轮廓的统计属性与几何属性值，显示如下：

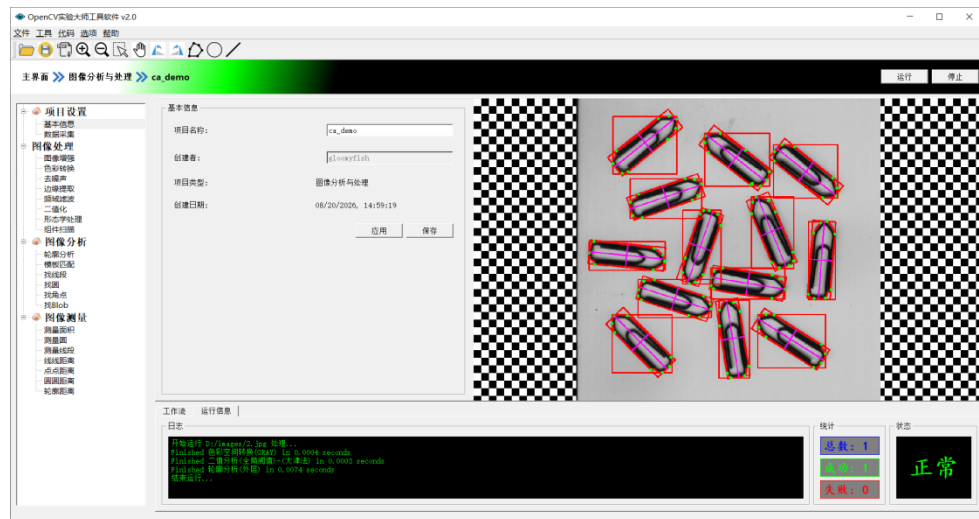
| 轮廓信息统计表 |                 |               |        |         |      |      |      |                     |        |        |       |
|---------|-----------------|---------------|--------|---------|------|------|------|---------------------|--------|--------|-------|
|         | 外接矩形(x,y,w,h)   | 中心(cx,cy)     | 周长     | 面积      | 圆度   | 凸度   | 纵横比  | 外接圆(cx,cy,r)        | 角度     | 长轴     | 短轴    |
| 1       | 81,489,136,131  | 148.84,555.69 | 417.67 | 7929.00 | 0.64 | 1.06 | 0.96 | 146.95,550.06,84.00 | 133.85 | 187.00 | 58.43 |
| 2       | 411,488,154,121 | 490.13,546.96 | 438.36 | 8306.00 | 0.62 | 1.07 | 0.79 | 484.86,545.90,87.00 | 123.48 | 199.40 | 56.63 |
| 3       | 323,456,69,176  | 355.90,549.04 | 438.84 | 7999.50 | 0.60 | 1.07 | 0.39 | 353.25,543.27,88.00 | 172.24 | 203.01 | 54.39 |
| 4       | 140,409,166,86  | 220.30,452.00 | 423.50 | 7999.00 | 0.64 | 1.07 | 0.52 | 222.84,456.46,84.00 | 106.69 | 188.03 | 56.33 |
| 5       | 305,383,168,70  | 385.78,417.62 | 416.94 | 8147.00 | 0.65 | 1.05 | 0.42 | 388.69,419.39,84.00 | 98.00  | 188.41 | 58.22 |
| 6       | 29,323,172,65   | 110.75,355.33 | 422.55 | 8017.50 | 0.62 | 1.05 | 0.38 | 114.59,357.66,86.00 | 96.45  | 190.05 | 56.38 |
| 7       | 526,278,61,176  | 555.62,370.24 | 433.97 | 7922.00 | 0.59 | 1.06 | 0.35 | 553.87,365.06,88.00 | 5.64   | 201.67 | 53.55 |
| 8       | 241,252,87,167  | 282.94,331.15 | 427.75 | 7774.50 | 0.62 | 1.08 | 0.52 | 278.43,335.48,85.00 | 17.60  | 189.15 | 54.52 |
| 9       | 397,222,96,162  | 444.43,305.38 | 433.30 | 8046.00 | 0.64 | 1.09 | 0.59 | 443.23,300.85,85.00 | 156.72 | 187.70 | 56.47 |
| 10      | 122,181,162,90  | 199.55,224.54 | 425.26 | 7945.50 | 0.64 | 1.07 | 0.56 | 205.50,228.50,85.00 | 70.92  | 189.59 | 55.66 |
| 11      | 447,104,145,121 | 518.38,163.29 | 419.87 | 8148.00 | 0.65 | 1.06 | 0.83 | 518.50,169.50,85.00 | 127.70 | 184.96 | 58.52 |

关闭<轮廓信息统计表>、在过滤设置中勾选周长，最小输入 400，最大输入 600，点击【应

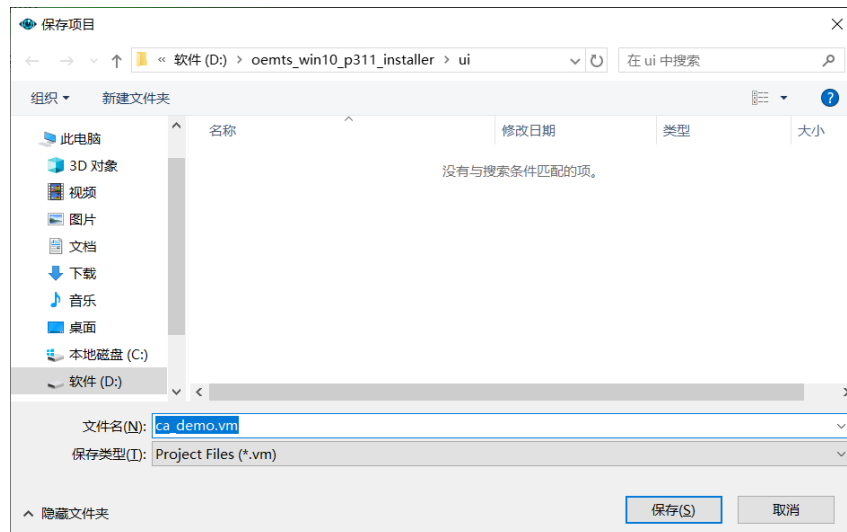
用】，然后再点击右上角【运行】按钮，显示如下：



点击左侧算子区域<基本信息>，在参数面板项目名称中输入 `ca_demo`、然后点击【应用】按钮，显示如下：



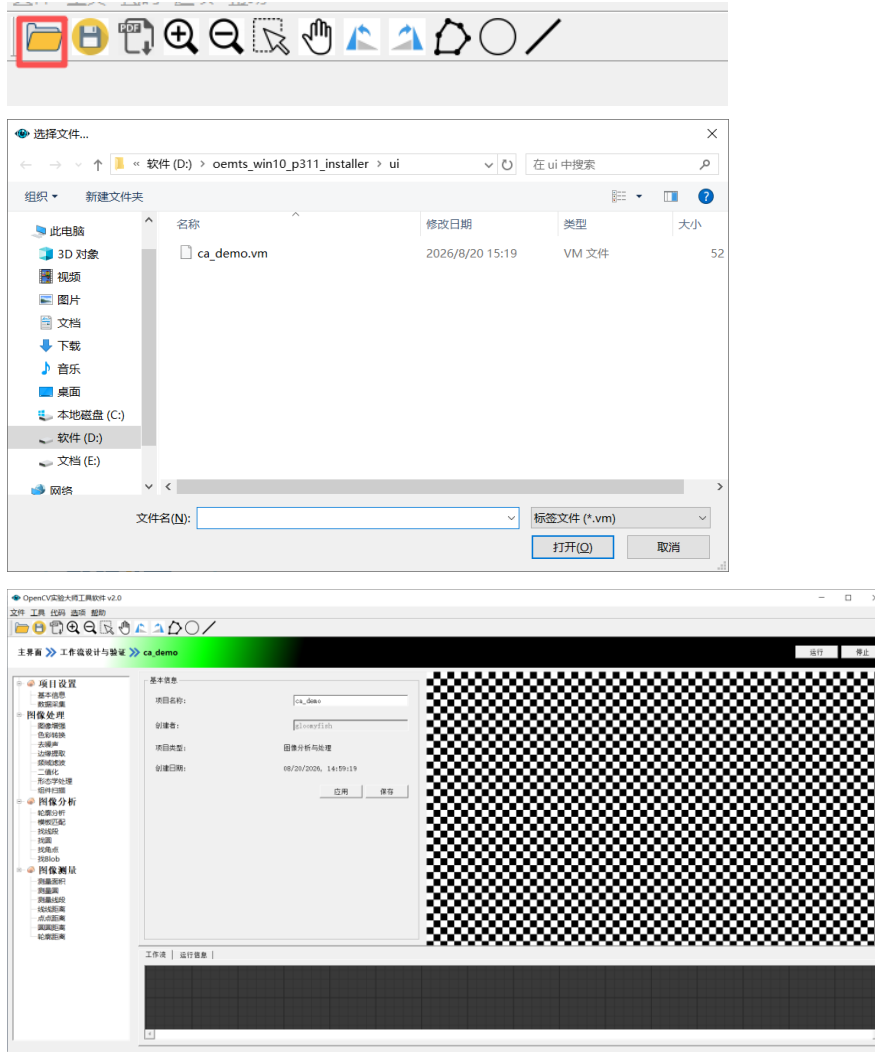
然后点击工具栏，保存图标，显示如下，可以保存 workflow。显示如下：



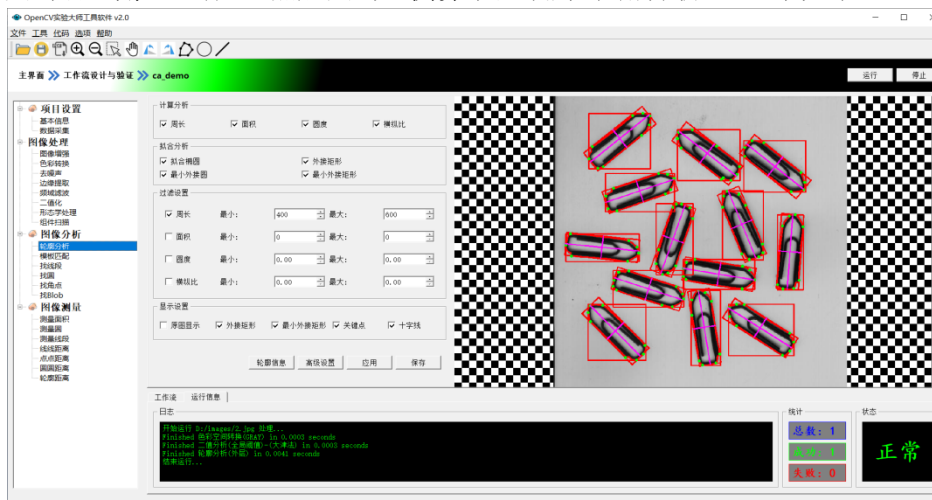
点击导航栏上的 主界面 文字 返回主界面

## 主界面 >>> 图像分析与处理 >>> ca\_demo

然后点击工具栏上的打开文件夹图标，选择刚刚保存好的 ca\_demo.vm 文件，打开如下：



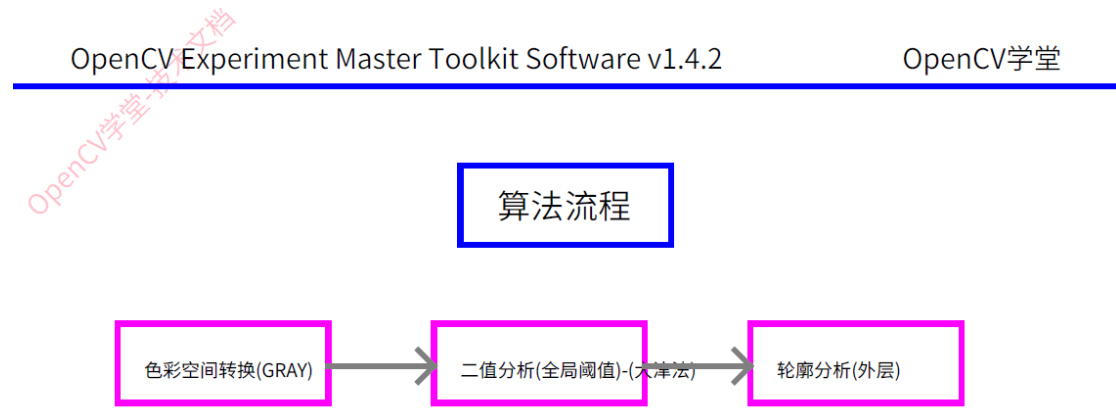
点击右上角【运行】，然后点击左侧算子区域的<轮廓分析>，显示如下：



这样我们就完成了一个基于 OpenCV 实验大师工具软件的算子设计、运行、保存、再打开。



运行以后，点击工具栏上的 PDF 图标，还可以导出该工作流对应的 PDF 版本文档说明，方便教学与技术交流使用。导出的 PDF 文档，截图示意如下：

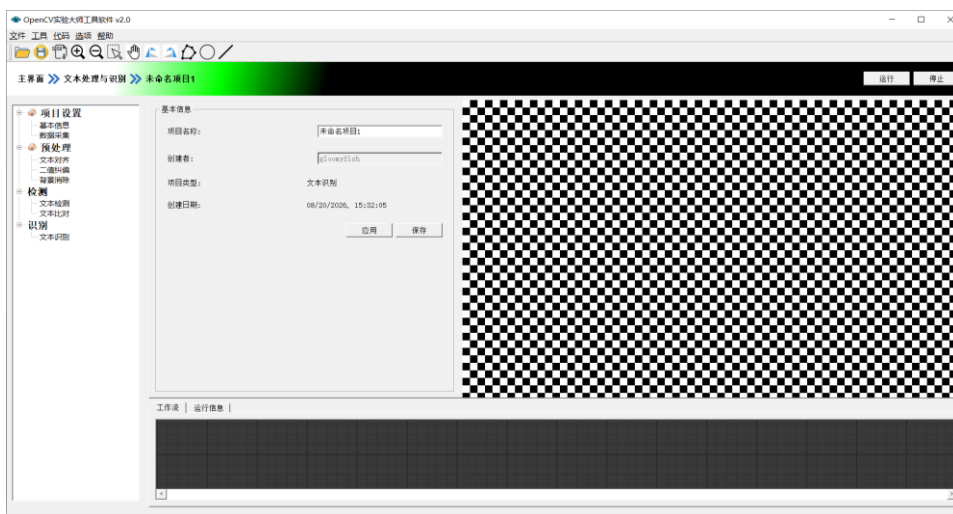


还可以导出相关 AI 提示词，这样方便 AI 给你自动生成对应的 OpenCV 代码：



### 三：文本处理与识别

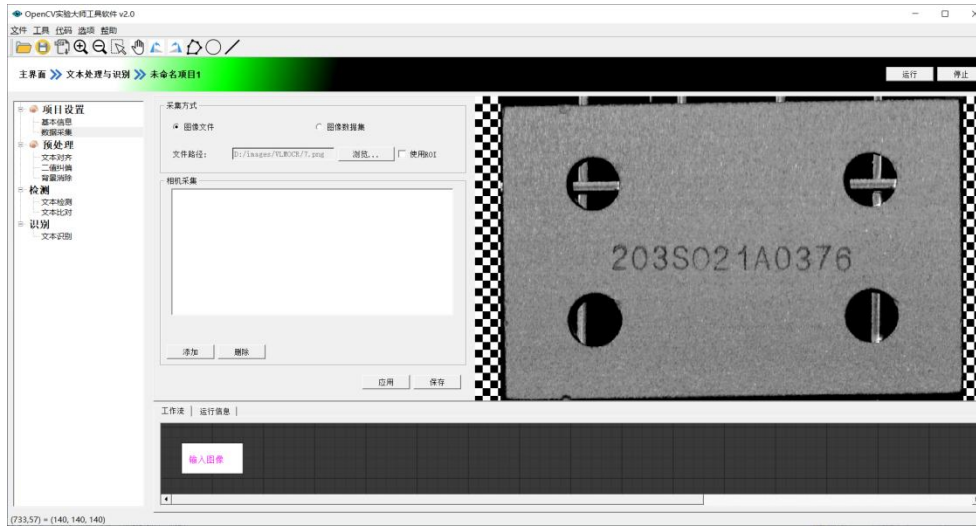
回到主界面，点击【文本处理与识别】，进入文本预处理、检测、OCR 识别模块，主界面显示如下：



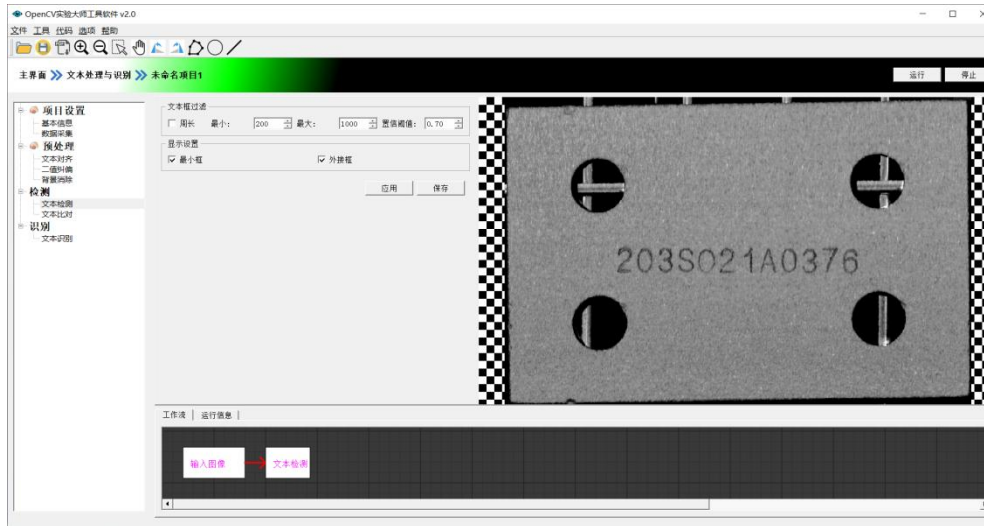
点击左侧算子栏-数据采集，导入一张图像（**注意：**目录路径不要有空格跟中文），点击【应



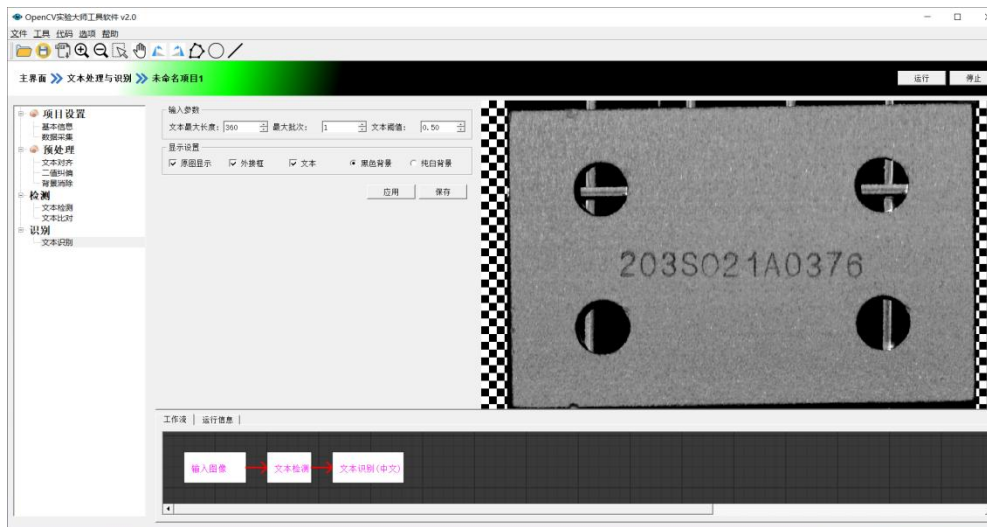
用】按钮，显示如下：



继续点击左侧算子栏<文本检测>，保存默认参数，直接点击【应用】按钮，显示如下：



继续点击左侧算子栏<文本识别>，保存默认参数，直接点击【应用】按钮，显示如下：



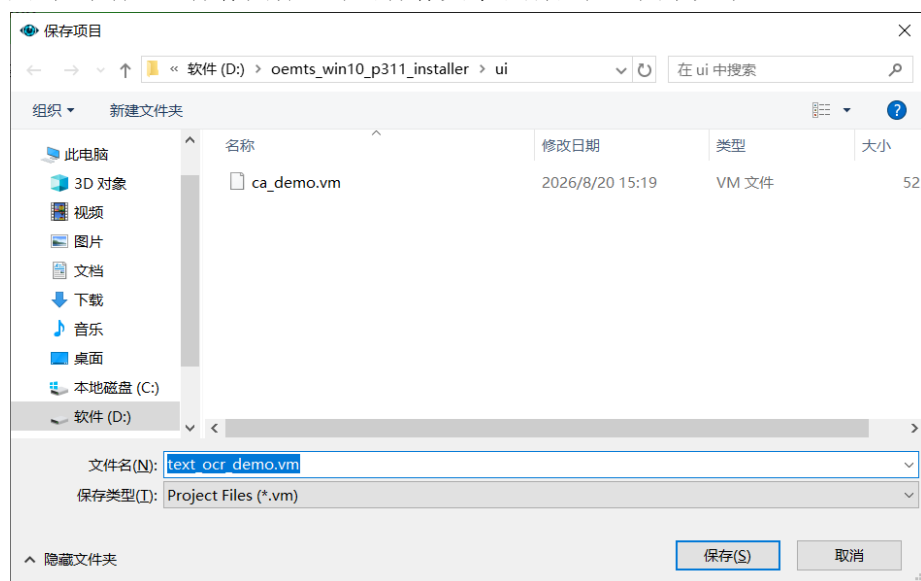
点击右上角【运行】按钮，显示效果如下：



在显示设置中，可以去掉勾选<原图显示>，你会直接看到识别后的文字输出。  
点击左侧算子栏-基本信息，输入 text\_ocr\_demo，然后点击【应用】按钮，显示如下：



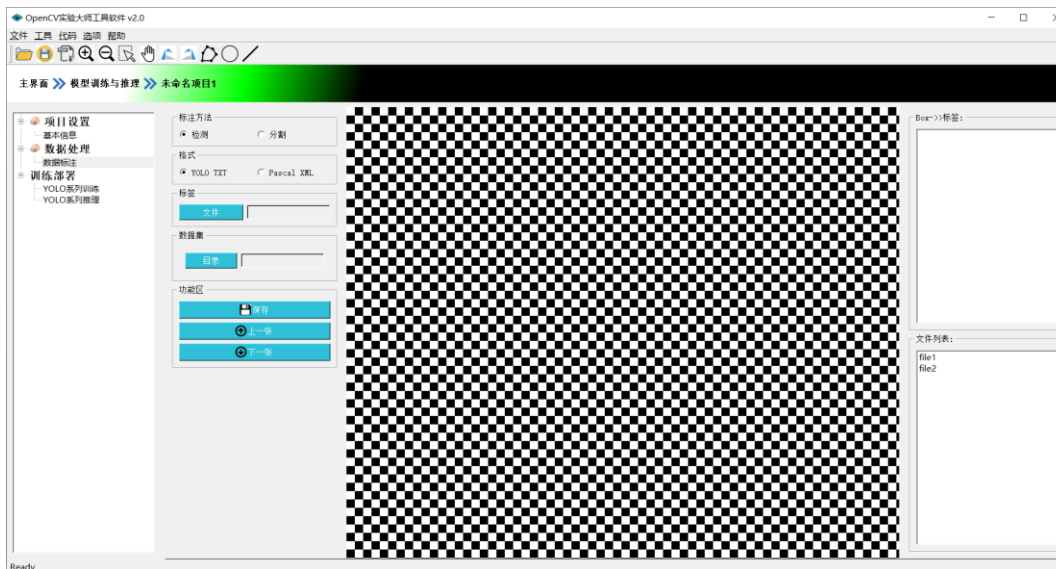
点击工具栏上 保存图标，即可保存文本识别流程，图示如下：



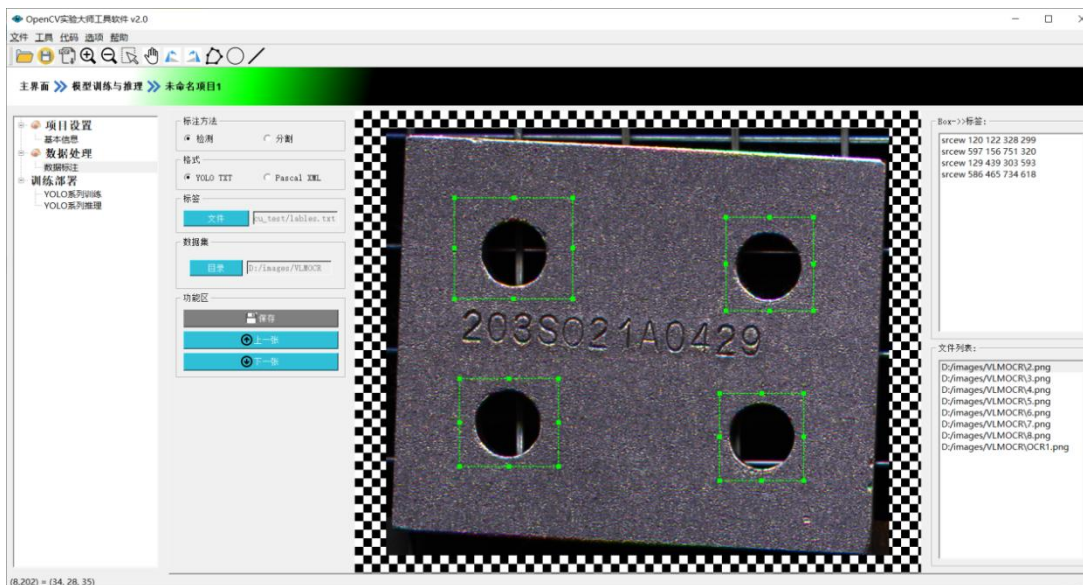
## 四：YOLO 深度学习训练与部署

### 1. 数据标注

支持对象检测与实例分割数据标注，保存标注数据。点击左侧【数据标注】，显示如下：



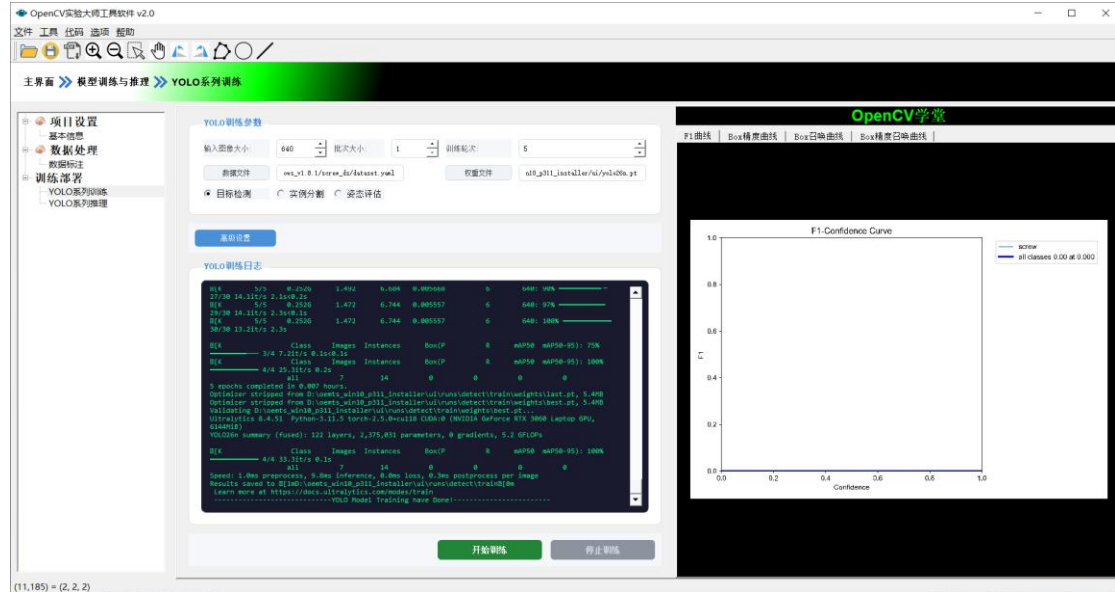
设置好标签文件、数据集目录(图像文件夹)、就可以直接用鼠标开始标注，标注完成之后，点击【保存】按钮，可以继续标注下一张图像，显示如下：



### 2. 模型训练

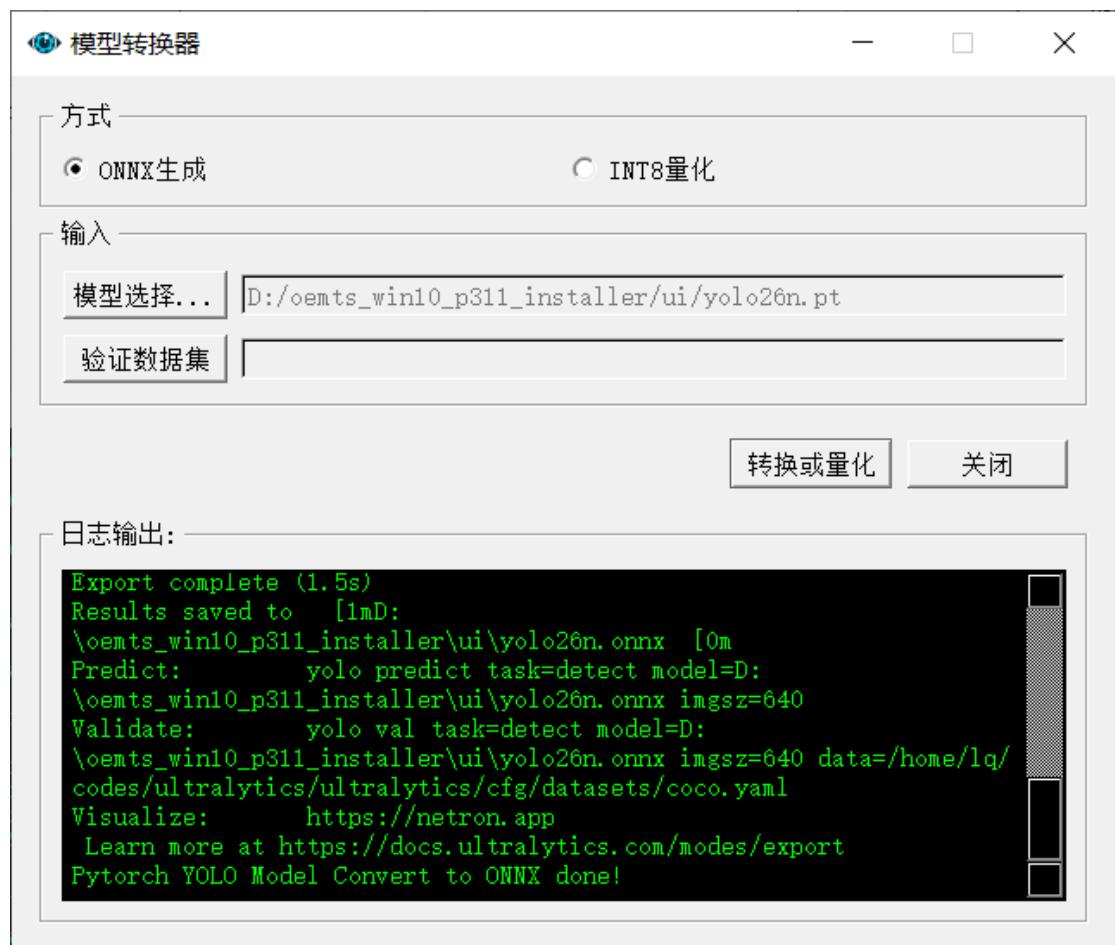
支持 YOLO 系列对象检测、实例分割、姿态评估的模型训练，选择数据文件（数据集的 yaml 描述文件路径）、权重文件（YOLO26 或者 YOLO11 的 pt 文件）、选择模型类型（对象检测、实例分割、姿态评估）。相关参数设置如下：





训练好的模型与日志文件默认保存路径为：D:\oemts\_win10\_p311\_installer\ui\runs\

从菜单栏 -> 工具 -> ONNX Converter 打开，选择训练好的 best.pt 文件，开启模型转换（从 pt 文件到 onnx 格式），运行显示如下：

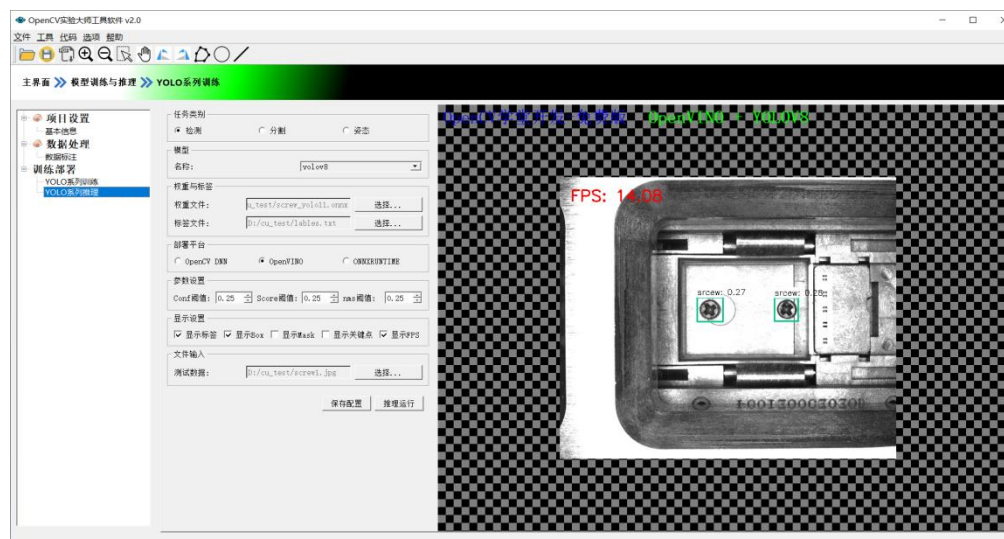


**注意：**如果选择 INT8 量化，输入的要求是 ONNX 格式模型，而且必须提供验证数据集的文件夹目录。才可以完成相关 YOLO 系列模型的 INT8 量化。

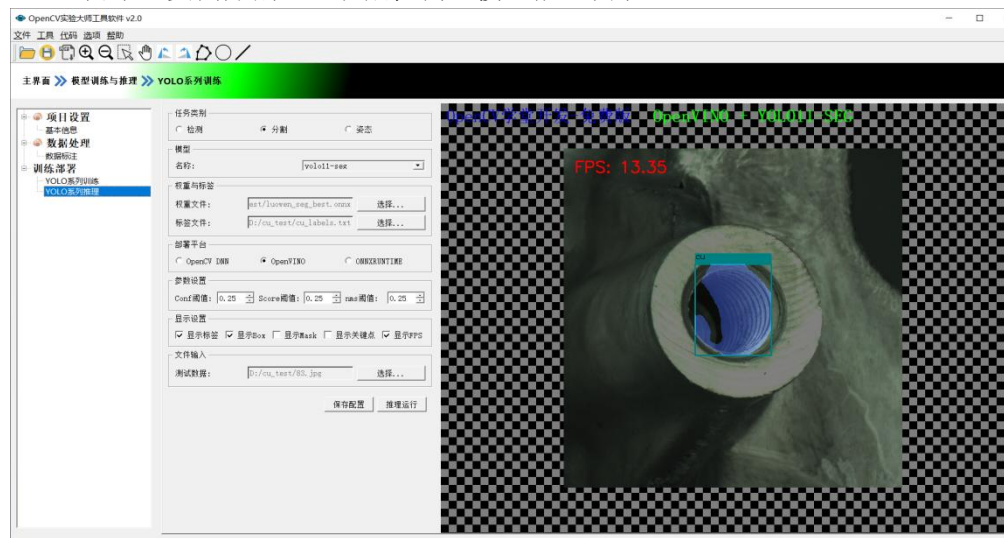


### 3. 模型推理支持

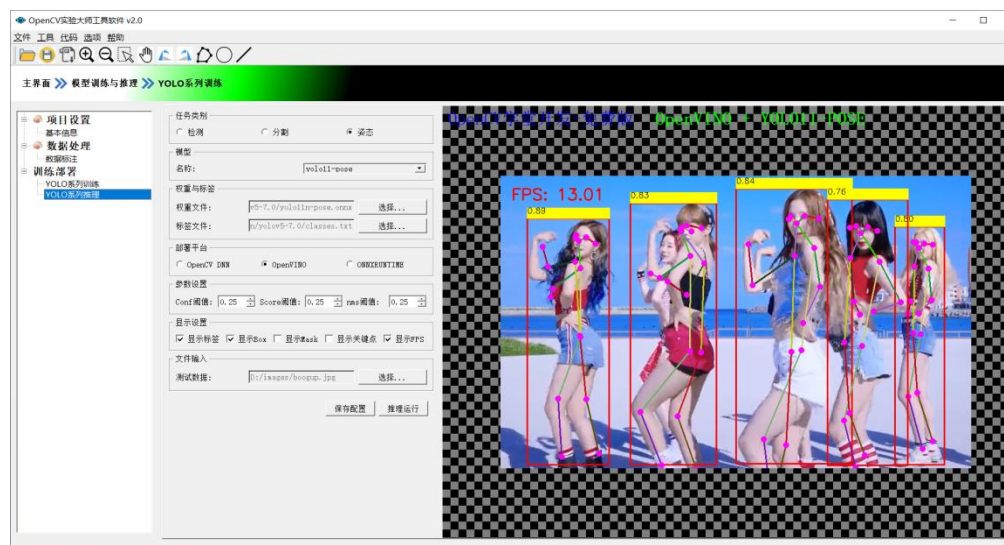
YOLO 对象检测模型，实现工业零件查找与定位



YOLO 系列 - 实例分割工业缺陷检测，模型推理演示：



YOLO 系列 - 姿态评估人体 17 个关键点检测演示

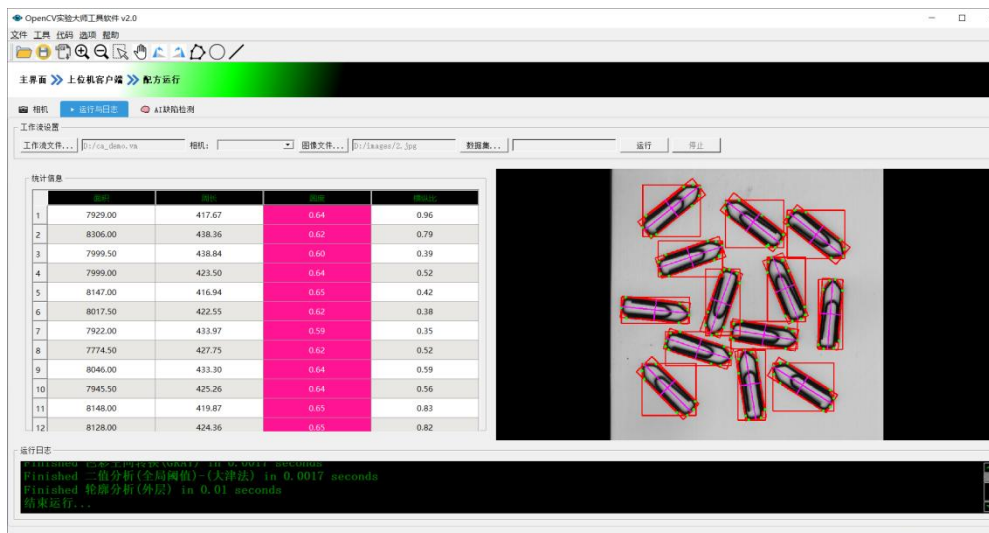


## 五：上位机集成部署复用

### 工业相机集成 – 示例演示

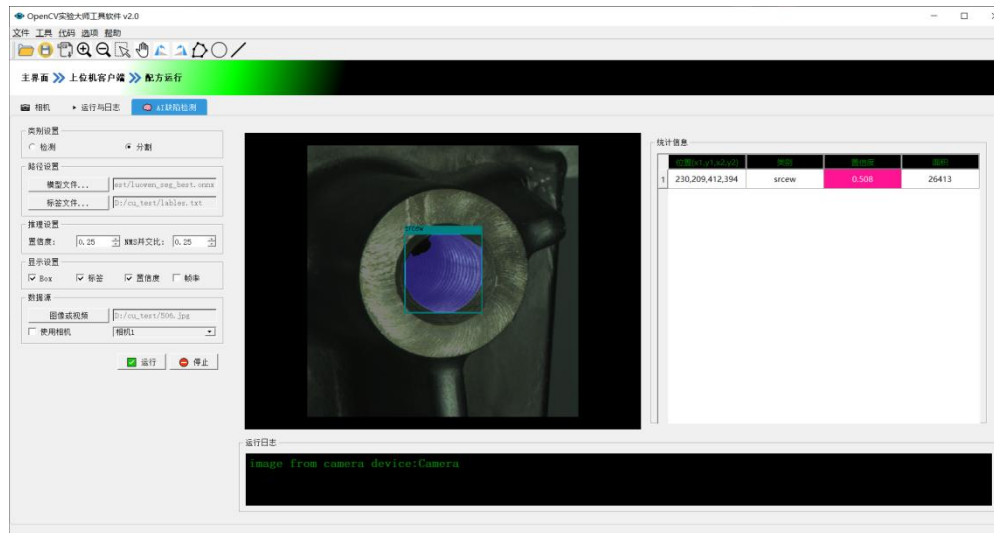


### 工作流复用 – 直接运行示例演示



### OCR 识别 – 工业相机集成演示实时 OCR 示例演示





## 六：SDK 集成测试演示

OpenCV 实验大师支持工作流 C++、C#、Python 语言的 SDK 调用与第三方集成，支持 YOLO 系列模型 C++部署三行代码即可调用。当前版本 SDK 基于 OpenCV4.10 完成。

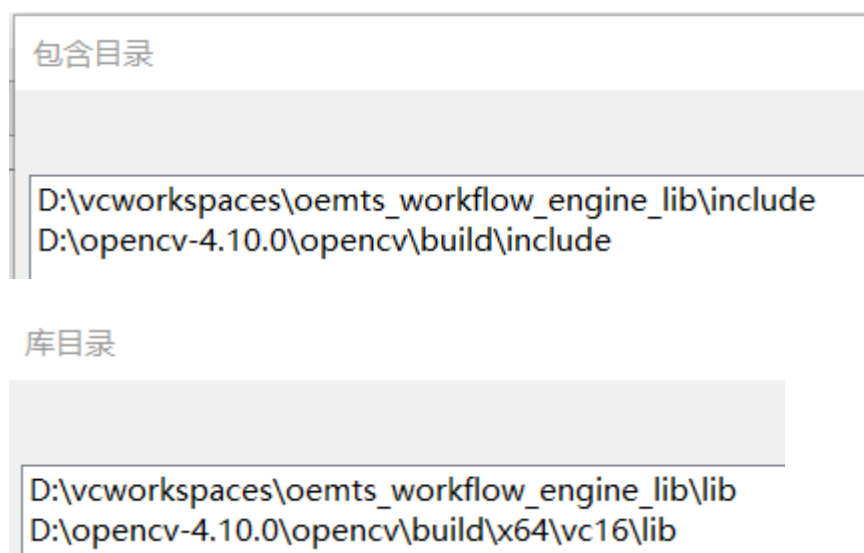
### OpenCV 实验大师工作流引擎仓库

[https://gitee.com/opencv\\_ai/opencv\\_workflow](https://gitee.com/opencv_ai/opencv_workflow)

### SDK 下载地址：

[https://gitee.com/opencv\\_ai/opencv\\_workflow/blob/master/oemts\\_workflow\\_engine\\_lib.zip](https://gitee.com/opencv_ai/opencv_workflow/blob/master/oemts_workflow_engine_lib.zip)

VS2022 中工作流引擎 C++ SDK 配置与调用演示（需要先下载 OpenCV4.10 并配置好开发环境支持）、然后配置好 SDK 的包含目录、库目录、图示如下：



相关 dll 依赖支持，推荐直接 copy 到 exe 执行目录下即可（你懂我在说什么）。



以模板匹配为例，可以直接引用构建好的 workflow 文件，然后通过 SDK 加载调用，代码如下：

```
#include "main_workflow.h"
#include <iostream>
#include <fstream>

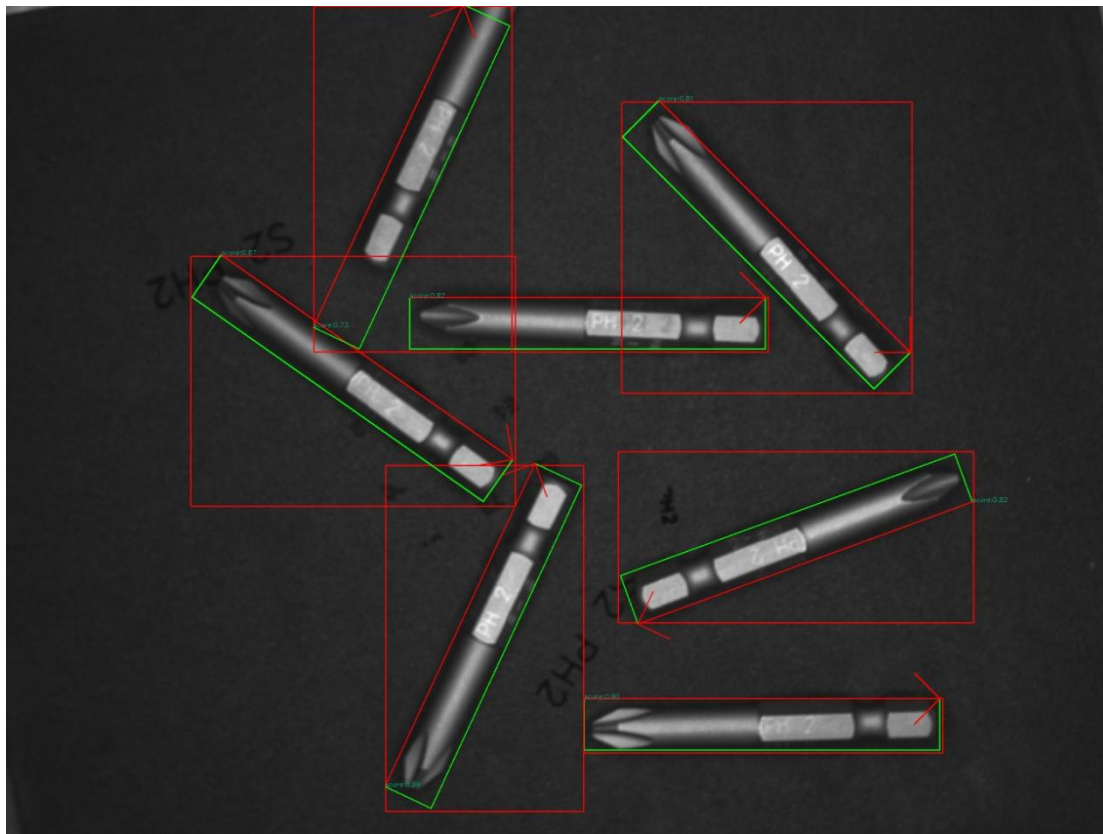
int main(int argc, char** argv) {
    // load opencv workflow sdk
    std::shared_ptr<QTongCoreCVWorkFlow> engine(new QTongCoreCVWorkFlow());
    bool succ = engine->initWorkFlow("D:/python/opencv_workflow/template_demo.vm",
                                     "OpenCVXUETANG-OEMTS-CPPSDK");
    if (!succ) {
        std::cout << "Could not load workflow file here..." << std::endl;
        return -1;
    }
    // image view
    cv::Mat frame = cv::imread("D:/python/opencv_workflow/target.bmp");
    cv::namedWindow("OpenCV学堂-边缘匹配", cv::WINDOW_NORMAL);
    cv::imshow("OpenCV学堂-边缘匹配", frame);

    // measure with caliper sdk
    cv::Mat result;
    std::vector<std::string> logs;
    engine->run_workflow(frame, result, logs);

    auto boxes = engine->getMatchedBoxInfos();

    cv::namedWindow("OpenCV学堂-边缘匹配结果", cv::WINDOW_NORMAL);
    cv::imshow("OpenCV学堂-边缘匹配结果", result);
    cv::imwrite("result.jpg", result);
    cv::waitKey(0);
    cv::destroyAllWindows();
    return 0;
}
```

运行结果如下：



SDK 调用分别是导入支持、初始化加载工作流，调用工作与显示返回数据

第一步，导入支持

```
#include "main_workflow.h"
```

第二步，加载工作流

```
// load opencv workflow sdk
std::shared_ptr<QTongCoreCVWorkFlow> engine(new QTongCoreCVWorkFlow());
bool succ = engine->initWorkFlow("D:/python/opencv_workflow/template_demo.vm",
                                   "OpenCVXUETANG-OEMTS-CPPSDK");
```

第三步，读取图像，调用工作流

```
cv::Mat frame = cv::imread("path/to/image");
cv::Mat result;
std::vector<std::string> logs;
engine->run_workflow(frame, result, logs);
```

第四步，读取返回数据与返回图像

```
// 读取模板匹配结果
auto boxes = engine->getMatchedBoxInfos();
// 读取轮廓分析的轮廓
auto contours = engine->getContours();
// 读取轮廓分析的统计结果
auto contours_stats = engine->getContourStatsInfo();
// 读取 BLOB 分析的结果
auto blobs = engine->getBlobInfos();
// 读取圆检测结果
auto circles = engine->getDetectCircles();
// 读取圆测量结果
auto circles = engine->getMeasureCircleInfo();
// 读取线段检测结果
auto lines = engine->getDetectLineInfos();
// 读取线段测量结果
auto lines = engine->getMeasureLineInfos();
```

YOLO 对象检测 SDK 支持，直接读取 ONNX 格式的 YOLO 系列模型文件，完成对象检测模型部署与推理，当前支持 YOLO 系列包含 YOLOv8\YOLO11\YOLO26 三个主流版本模型部署。

两个支持 SDK 分别是 [YOLO11Detector](#) 与 [YOLO26Detector](#)，调用方式完全一致。[YOLO11Detector](#) 支持 YOLOv8~YOLO11 对象检测推理，[YOLO26Detector](#) 只支持 YOLO26 的非 NMS 模式导出推理，NMS 导出模式推理请用前一个。

SDK 层面唯一的依赖是 OpenCV 相关的库，此外不依赖任何 C++库，当前依赖版本 OpenCV4.10。

YOLOv8 对象检测模型，通过 SDK 部署示例如下，只需要几行代码即可完成。

```
#include "yolo11_detector.h"
#include <iostream>
#include <fstream>

int main(int argc, char** argv) {
    std::string mode_path = "D:/python/yolov5-7.0/yolov8n.onnx";
    std::string label_path = "D:/python/yolov5-7.0/classes.txt";
    std::shared_ptr<YOLO11Detector> detector(new YOLO11Detector());
    cv::Mat image = cv::imread("D:/mytest.png");
    detector->initConfig(mode_path, label_path, 0.25);
    detector->infer(image);
    cv::imshow("OpenCV实验大师SDK+YOLOv8对象检测", image);
    cv::waitKey(0);
    cv::destroyAllWindows();
    return 0;
}
```

代码运行结果如下，调用 SDK，只需要三行代码即可完成，分别是创建对象实例、初始化、推理，然后通过 `auto items = detector->getDetectResults();` 可以获取所有检测结果。图示如下：



我们的愿景

Make OpenCV Development Easy

OpenCV 实验大师的产品文化

开源 免费 协作 共创